



Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251218
Nama Lengkap	Agatha Sabda Alethea Prabawa
Minggu ke / Materi	15 / Fungsi Rekursif

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Pada bagian ini, tuliskan kembali semua materi yang telah anda pelajari minggu ini. Sesuaikan penjelasan anda dengan urutan materi yang telah diberikan di saat praktikum. Penjelasan anda harus dilengkapi dengan contoh, gambar/ilustrasi, contoh program (source code) dan outputnya. Idealnya sekitar 5-6 halaman.

Pengertian Rekursif

Fungsi rekursif adalah sebuah fungsi yang di dalam tubuhnya terdapat pemanggilan terhadap dirinya sendiri. Dengan kata lain, fungsi ini mendefinisikan dirinya sendiri secara berulang hingga suatu kondisi tertentu terpenuhi. Konsep rekursif ini sebenarnya berasal dari dunia matematika, di mana banyak rumus atau definisi matematika yang dinyatakan secara rekursif, yaitu sebuah fungsi yang nilainya ditentukan berdasarkan nilai fungsi itu sendiri pada argumen yang lebih kecil.

Secara sederhana, fungsi rekursif dapat dipahami sebagai fungsi yang menyelesaikan suatu masalah dengan cara memecah masalah tersebut menjadi bagian-bagian yang lebih kecil, lalu memanggil dirinya sendiri untuk menyelesaikan masing-masing bagian tersebut. Proses pemecahan ini terus berlangsung hingga masalah yang dihadapi sudah cukup kecil dan sederhana untuk diselesaikan secara langsung tanpa perlu pemecahan lebih lanjut.

Karena fungsi rekursif memanggil dirinya sendiri secara berulang, sangat penting untuk memastikan bahwa fungsi tersebut memiliki kondisi yang jelas untuk berhenti. Tanpa kondisi berhenti yang tepat, fungsi akan terus memanggil dirinya sendiri tanpa henti dan pada akhirnya akan menyebabkan program mengalami hang up atau kehabisan memori. Oleh karena itu, penggunaan fungsi rekursif harus dilakukan dengan cermat dan hati-hati.

Di dalam sebuah fungsi rekursif, terdapat dua bagian penting yang wajib ada. Bagian pertama disebut Base Case, yaitu kondisi yang menentukan kapan fungsi rekursif harus berhenti. Base case merupakan kasus paling sederhana dari suatu masalah yang dapat diselesaikan secara langsung tanpa perlu pemanggilan rekursif lagi. Bagian kedua disebut Recursive Case, yaitu

bagian yang berisi pernyataan pemanggilan fungsi itu sendiri dengan argumen yang berbeda (biasanya lebih kecil), dan akan terus diulang hingga kondisi base case tercapai.

Kedua bagian tersebut harus selalu hadir secara bersamaan dalam setiap fungsi rekursif yang dibuat. Jika base case tidak ada atau tidak terdefinisi dengan benar, maka fungsi tidak akan pernah berhenti. Sebaliknya, jika recursive case tidak ada, maka fungsi tersebut bukan lagi fungsi rekursif melainkan fungsi biasa.

Kelebihan dan Kekurangan

Seperti halnya teknik pemrograman lainnya, fungsi rekursif memiliki sisi kelebihan maupun kekurangan yang perlu dipertimbangkan sebelum digunakan dalam sebuah program.

Kelebihan fungsi rekursif antara lain adalah sebagai berikut:

1. Kode program yang dihasilkan cenderung lebih singkat, bersih, dan elegan dibandingkan solusi iteratif untuk permasalahan yang sama. Hal ini membuat kode lebih mudah dibaca dan dipahami dari sisi logika algoritma.
2. Masalah yang kompleks dapat dipecah menjadi sub-masalah yang lebih kecil dan lebih mudah diselesaikan secara terstruktur. Pendekatan ini sangat cocok untuk masalah-masalah yang secara alami memiliki struktur rekursif, seperti penelusuran pohon, pengurutan data, dan berbagai masalah matematika.

Adapun kekurangan atau kelemahan dari fungsi rekursif adalah sebagai berikut:

3. Konsumsi memori yang lebih besar dibandingkan solusi iteratif. Setiap kali fungsi memanggil dirinya sendiri, Python akan menyimpan informasi konteks pemanggilan tersebut di dalam stack memori. Semakin dalam tingkat rekursi, semakin banyak memori yang dibutuhkan.
4. Efisiensi dan kecepatan eksekusi program cenderung lebih rendah dibandingkan solusi iteratif, terutama untuk kasus rekursi yang sangat dalam. Hal ini disebabkan oleh overhead tambahan yang terjadi setiap kali fungsi memanggil dirinya sendiri.

5. Fungsi rekursif seringkali lebih sulit untuk dilakukan proses debugging (pencarian dan perbaikan kesalahan) karena alur eksekusinya yang tidak linier dan melibatkan banyak lapisan pemanggilan yang saling bertumpuk.

Mengingat kelebihan dan kekurangan tersebut, penggunaan fungsi rekursif sebaiknya dipertimbangkan dengan matang. Rekursif lebih disarankan digunakan ketika masalah yang dihadapi secara alami bersifat rekursif dan keindahan serta kesederhanaan kode lebih diprioritaskan daripada efisiensi memori dan kecepatan eksekusi.

Bentuk Umum dan Studi Kasus

Secara umum, sebuah fungsi rekursif dalam Python ditulis dengan mendefinisikan sebuah fungsi menggunakan kata kunci `def`, kemudian di dalam tubuh fungsi tersebut terdapat baris kode yang memanggil kembali nama fungsi itu sendiri dengan argumen yang berbeda. Pola dasar ini berlaku untuk semua jenis fungsi rekursif, meskipun implementasi detailnya akan berbeda-beda tergantung pada masalah yang diselesaikan.

Perlu dipahami pula bahwa pada dasarnya setiap fungsi rekursif selalu memiliki padanan solusi iteratifnya. Artinya, semua masalah yang dapat diselesaikan secara rekursif juga dapat diselesaikan menggunakan perulangan (loop) biasa. Perbedaannya hanya terletak pada pendekatan dan gaya penulisan kode.

Sebagai studi kasus untuk memahami konsep fungsi rekursif, akan digunakan perhitungan faktorial. Faktorial adalah operasi matematika yang menghitung hasil perkalian dari semua bilangan bulat positif mulai dari 1 hingga n . Secara iteratif, perhitungan faktorial dilakukan dengan menyiapkan variabel penampung, kemudian melakukan perulangan dari 1 hingga n sambil terus mengalikan nilai penampung tersebut dengan nilai iterasi yang sedang berjalan.

Namun secara rekursif, faktorial dapat didefinisikan dengan lebih ringkas menggunakan dua kondisi. Kondisi pertama adalah base case, yaitu apabila n bernilai 0 atau 1 maka faktorialnya adalah 1. Kondisi kedua adalah recursive case, yaitu untuk nilai n yang lebih besar dari 1, faktorial

dari n adalah n dikalikan dengan faktorial dari $n-1$. Dengan definisi ini, fungsi akan terus memanggil dirinya sendiri dengan nilai n yang semakin mengecil hingga mencapai base case.

Sebagai ilustrasi cara kerja fungsi faktorial rekursif, misalkan kita memanggil faktorial(4). Berikut adalah alur prosesnya secara bertahap:

6. Pemanggilan pertama: faktorial(4) belum bisa diselesaikan langsung, sehingga memanggil faktorial(3).
7. Pemanggilan kedua: faktorial(3) belum bisa diselesaikan langsung, sehingga memanggil faktorial(2).
8. Pemanggilan ketiga: faktorial(2) belum bisa diselesaikan langsung, sehingga memanggil faktorial(1).
9. Pemanggilan keempat: faktorial(1) mencapai base case dan langsung mengembalikan nilai 1.
10. Nilai 1 dikembalikan ke pemanggilan ketiga, menghasilkan $2 \times 1 = 2$.
11. Nilai 2 dikembalikan ke pemanggilan kedua, menghasilkan $3 \times 2 = 6$.
12. Nilai 6 dikembalikan ke pemanggilan pertama, menghasilkan $4 \times 6 = 24$ sebagai jawaban akhir.

Dari ilustrasi di atas dapat terlihat dengan jelas pola kerja rekursif, yaitu fungsi terus menerus memanggil dirinya sendiri dengan argumen yang semakin kecil (recursive case) hingga akhirnya mencapai kondisi yang cukup sederhana untuk diselesaikan langsung (base case), kemudian nilai-nilai tersebut dikembalikan satu per satu ke atas hingga diperoleh jawaban akhir.

Konsep yang sama dapat diterapkan pada berbagai permasalahan lain yang memiliki struktur berulang dan dapat dipecah menjadi sub-masalah yang lebih kecil, seperti perhitungan deret Fibonacci, konversi basis bilangan, perhitungan permutasi dan kombinasi, serta berbagai algoritma pengurutan dan penelusuran data. Kunci keberhasilan dalam membuat fungsi rekursif

yang benar adalah dengan mengidentifikasi base case dan recursive case secara tepat sebelum mulai menulis kode.

Dengan memahami konsep dasar rekursif yang telah dijabarkan melalui studi kasus faktorial ini, diharapkan pembaca dapat lebih mudah mengembangkan dan mengimplementasikan fungsi rekursif untuk berbagai permasalahan pemrograman yang lebih kompleks di kemudian hari.

BAGIAN 2: LATIHAN MANDIRI (60%)

Link github: https://github.com/gathasja/71251218_Agatha.git

SOAL 1

```
1 def bilprima(n, cek=2):
2     if n < 2:
3         return False
4     if cek == n:
5         return True
6     if n % cek == 0:
7         return False
8     return bilprima(n, cek + 1)
9
10 inpute = int(input("bilangan: "))
11
12 if bilprima(inpute):
13     print(f"{inpute} adalah prima")
14 else:
15     print(f"{inpute} bukan prima")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
PS D:\KULIAH\MATA KULIAH SEMESTER 2\PrakAlPr
kAlPro-71251218/minggu15/soal01.py"
bilangan: 8
8 bukan prima
PS D:\KULIAH\MATA KULIAH SEMESTER 2\PrakAlPr
kAlPro-71251218/minggu15/soal01.py"
bilangan: 5
5 adalah prima
```

Penjelasan soal01:

1. `def bilprima(n, cek=2):` -> Membuat fungsi, cek dimulai dari 2.
2. `if n < 2:`
`return False` -> Jika n kurang dari 2, maka bilangan tersebut bukan prima (return false).
3. `if cek == n:`
`return True` -> Jika pembagi sudah mencapai nilai n, maka bilangan tersebut prima (return true).
4. `if n % cek == 0:`
`return False` -> Jika n habis dibagi cek, maka bilangan tersebut bukan prima (return false).
5. `return bilprima(n, cek + 1)` -> Rekursi, jika belum ketemu pembagi maka program akan mengulangi proses yang sama dengan mengecek angka berikutnya.
6. `inpute = int(input("bilangan: "))` -> Meminta user untuk memberikan bilangan yang akan di cek.
7. `if bilprima(inpute):`
`print(f"{inpute} adalah prima`
`else:`
`print(f"{inpute} bukan prima")`
-> Memanggil fungsi bilprima dan mengecek apakah inputan yang diberi user adalah bilangan prima atau bukan. Lalu hasilnya akan ditampilkan.

SOAL 2

```
1 def palindrom(teks):
2     if len(teks) <= 1:
3         return True
4     if teks[0] != teks[-1]:
5         return False
6     return palindrom(teks[1:-1])
7
8 def rapi(teks):
9     teks = teks.lower()
10
11     teksrapi = ""
12     for karakter in teks:
13         if 'a' <= karakter <= 'z':
14             teksrapi += karakter
15     return teksrapi
16
17 kalimat = input("isi kalimat: ")
18 kalimat_bersih = rapi(kalimat)
19
20 if palindrom(kalimat_bersih):
21     print(f"{kalimat} adalah palindrom")
22 else:
23     print(f"{kalimat} bukan palindrom")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS D:\KULIAH\MATA KULIAH SEMESTER 2\PrakAlPro> & C
ro/PrakAlPro-71251218/minggu15/soal02.py"
isi kalimat: kasur rusak
kasur rusak adalah palindrom

Penjelasan soal02:

1. `def palindrom(teks):` -> Mendefinisikan fungsi rekursif.
2. `if len(teks) <= 1:`
3. `return True` -> Base case 1: jika tersisa 0 atau 1 karakter, berarti palindrom.
4. `if teks[0] != teks[-1]:`
5. `return False` -> Base case 2: jika karakter awal tidak sama dengan akhir, berarti bukan palindrom.
6. `return palindrom(teks[1:-1])` -> Rekursif, hapus ujung, cek lagi.
7. `def rapi(teks):`
8. `teks = teks.lower()` -> Ubah semua huruf menjadi kecil.
9. `teksrapi = ""` -> String kosong untuk hasil.
10. `for karakter in teks:` -> Perulangan setiap karakter.
11. `if 'a' <= karakter <= 'z':` -> Mengambil huruf a-z.
12. `teksrapi += karakter` -> Tambah ke hasil.
13. `return teksrapi` -> Menampilkan teks yang sudah rapi.
14. `kalimat = input("isi kalimat: ")` -> Minta input dari user.
15. `kalimat_bersih = rapi(kalimat)` -> Memanggil fungsi rapi()
16. `if palindrom(kalimat_bersih):` -> Memanggil fungsi palindrom untuk mengecek kalimat dari user.
17. `print(f"{kalimat} adalah palindrom")` -> Jika true, maka program akan menampilkan kalimat tersebut adalah palindrom.
18. `else:`
19. `print(f"{kalimat} bukan palindrom")` -> Jika false, maka kalimat tersebut akan ditampilkan bukan palindrom.

SOAL 3

```
1 def deret(n):
2     if n <= 1:
3         return 1
4     if n % 2 == 0:
5         n = n - 1
6     return n + deret(n - 2)
7 try:
8     n = int(input("bilangan terakhir: "))
9     if n % 2 == 0:
10        n_asal = n
11        n = n - 1
12
13    hasil = deret(n)
14
15    print(f"deret ganjil: ", end="")
16    for i in range(1, n + 1, 2):
17        if i < n:
18            print(f"{i} + ", end="")
19        else:
20            print(f"{i}", end="")
21    print(f"\nhasil: {hasil}")
22 except ValueError:
23    print("harus berupa bilangan bulat!")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS D:\KULIAH\MATA KULIAH SEMESTER 2\PrakAlPro> & C:/Users/lenc
ro/PrakAlPro-71251218/minggu15/soal03.py
bilangan terakhir: 20
deret ganjil: 1 + 3 + 5 + 7 + 9 + 11 + 13 + 15 + 17 + 19
hasil: 100

Penjelasan soal03:

1. Fungsi deret(n) -> Fungsi rekursif untuk menjumlah bilangan ganjil dari 1 sampai n. Jika $n \leq 1$ kembalikan 1. Jika n genap, ubah jadi ganjil dengan $n-1$. Jika ganjil, jumlahkan n dengan hasil pemanggilan fungsi deret($n-2$).
2. Blok try -> Mencoba menjalankan program, menangani error jika input bukan angka.
3. Bagian input dan konversi -> Minta user memasukkan n. Jika n genap, turunkan 1 agar menjadi ganjil.

```
n = int(input("bilangan terakhir: "))
if n % 2 == 0:
    n_asal = n
    n = n - 1
```

4. Panggil fungsi deret(n) untuk mendapatkan jumlah total deret ganjil. Gunakan loop for i in range(1, n+1, 2) untuk mencetak bilangan ganjil dari 1 sampai n. Setiap bilangan dicetak dengan tanda + di antaranya, kecuali bilangan terakhir. Loop ini berlangkah 2 sehingga hanya bilangan ganjil yang dicetak. Lalu total jumlah deret yang sudah dihitung oleh fungsi rekursif akan ditampilkan.

```
hasil = deret(n)
print(f"deret ganjil: ", end="")
for i in range(1, n + 1, 2):
    if i < n:
        print(f"{i} + ", end="")
    else:
        print(f"{i}", end="")
print(f"\nhasil: {hasil}")
```

5. Jika user memasukkan huruf atau selain angka, tampilkan pesan harus berupa bilangan bulat!.

```
except ValueError:
    print("harus berupa bilangan bulat!")
```

SOAL 4

```
1 def jumlah_digit(n):
2     n = abs(n)
3     if n < 10:
4         return n
5     return (n % 10) + jumlah_digit(n // 10)
6
7 angka = int(input())
8 print(f"jumlah dari {angka} = {jumlah_digit(angka)}")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS D:\KULIAH\MATA KULIAH SEMESTER 2\PrakAIPro> & C:/Users/lenov
ro/PrakAIPro-71251218/minggu15/soal04.py"
123
jumlah dari 123 = 6
```

Penjelasan soal04:

1. `def jumlah_digit(n):` -> Mendefinisikan fungsi `jumlah_digit` dengan parameter `n`.
2. `n = abs(n)` -> Mengambil **nilai mutlak** dari `n` agar bilangan negatif bisa di proses.
3. `if n < 10:`
`return n` -> Jika `n` kurang dari 10, maka `n` itu sendiri yang akan ditampilkan.
4. `return (n % 10) + jumlah_digit(n // 10)` -> modulo 10 untuk mengambil digit terakhir, floor division 10 untuk membuang digit terakhir, `jumlah_digit()` untuk memanggil fungsi lagi untuk sisa bilangan. Jumlahkan digit terakhir dengan hasil rekursi.
5. `angka = int(input())` -> Minta input bilangan ke user.
6. `print(f"jumlah dari {angka} = {jumlah_digit(angka)}")` -> Menampilkan hasil.

SOAL 5

```
1 def kombinasi(n, r):
2     if r > n:
3         return 0
4     if r == 0 or r == n:
5         return 1
6     return kombinasi(n - 1, r - 1) + kombinasi(n - 1, r)
7
8 isin = input("n = ")
9 isir = input("r = ")
10 print(kombinasi(int(isin), int(isir)))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS D:\KULIAH\MATA KULIAH SEMESTER 2\PrakAlPro> & C:/Users/lenovo//ro/PrakAlPro-71251218/minggu15/soal05.py"

n = 5
r = 2
10

PS D:\KULIAH\MATA KULIAH SEMESTER 2\PrakAlPro> █

Penjelasan soal05:

1. `def kombinasi(n, r):` -> Fungsi kombinasi dengan 2 parameter.
2. `if r > n:`
3. `return 0` -> Jika $r > n$ maka akan menghasilkan 0 (tidak bisa dicari).
4. `if r == 0 or r == n:`
5. `return 1` -> jika $r == 0$ atau $r == n$ maka akan menghasilkan 1 cara saja.
6. `return kombinasi(n - 1, r - 1) + kombinasi(n - 1, r)` -> Melakukan rekursi dengan memanggil fungsinya sendiri dan menghitung hingga kondisi $r > n$, $r == 0$, atau $r == n$.
7. `isin = input("n = ")` -> Meminta user mengisi nilai n.
8. `isir = input("r = ")` -> Meminta user mengisi nilai r.
9. `print(kombinasi(int(isin), int(isir)))` -> menampilkan hasil perhitungan kombinasi.